

Preparing to IEEEExtreme20.0

Data Structures



Agenda

- **What is data structure**
- **Primitive and non primitive**
- **Linear and non Linear**
- **Data strucutre in action**

Two Main Categories

Primitive

Store data of only one type : the language's built-in types.

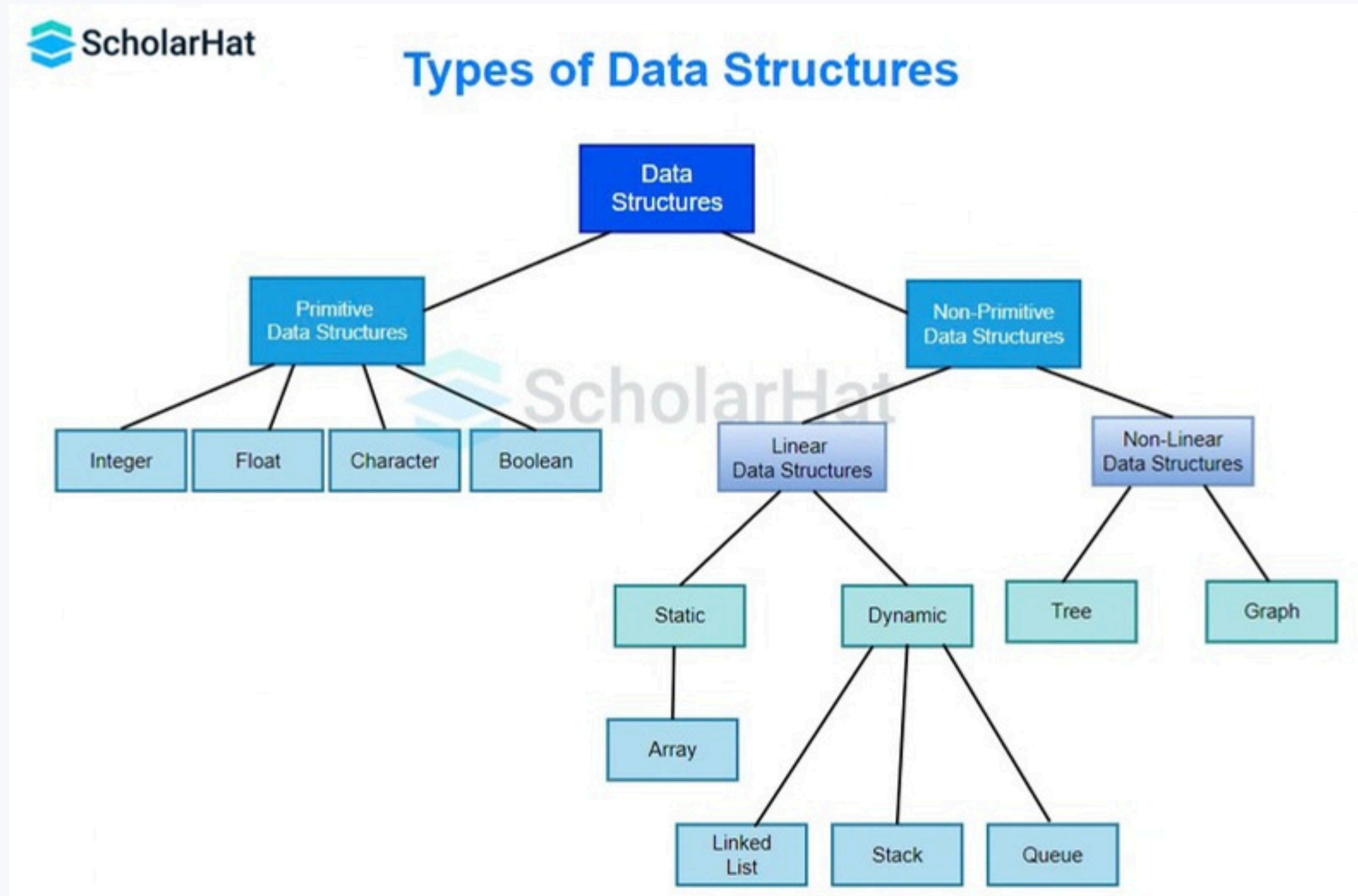
- Integer, float, character, boolean
- Fixed size, hardware-supported
- Fast and memory-efficient

Non-Primitive

Built from primitive types to hold data of more than one type.

- Arrays, linked lists, trees, graphs
- Linear: sequential, one after another
- Non-Linear: hierarchical connections

Two Main Categories



The Building Blocks

Directly supported by hardware, with fixed size and simple operations.

1

Simple

Represent single values, not derived from other structures.

2

Predefined

Built into most languages, handled directly by the CPU.

3

Efficient

Memory-efficient : simple operations, direct storage.

ADVANTAGES

- Efficient : fast, low memory overhead
- Easy to use : no complex relationships
- Fixed size : predictable memory usage

DISADVANTAGES

- Limited functionality basic types only
- No relationships between data elements

Six Core Types

I Integer

```
int a = 5;
```

Whole numbers, positive or negative.

F Float

```
float b = 5.75;
```

Numbers with a fractional / decimal part.

C Character

```
char c = 'A';
```

A single letter, digit, or symbol.

B Boolean

```
bool isActive = true;
```

One of two values: true or false.

S String

```
string n = "John";
```

A sequence of characters used as text.

P Pointer

```
int *ptr = &a;
```

Stores the memory address of a variable.

Linear vs. Non-Linear

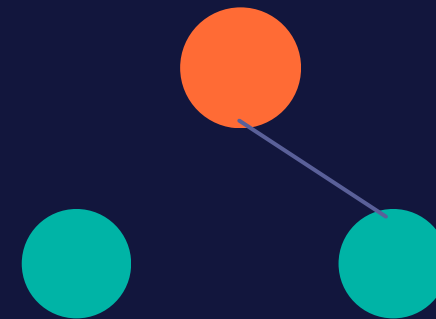
Non-primitive structures are built from primitive types and split by how their elements connect.



Linear

Elements arranged in sequence — each one connects to the previous and next. Single-pass traversal.

Array · Stack · Queue · Linked List · Deque



Non-Linear

Elements form a hierarchy with multiple ways to connect. Cannot be traversed in a single pass.

Tree · Graph · Hash Table

Arrays & Linked Lists

Two ways to hold a sequence of elements in memory.



Array

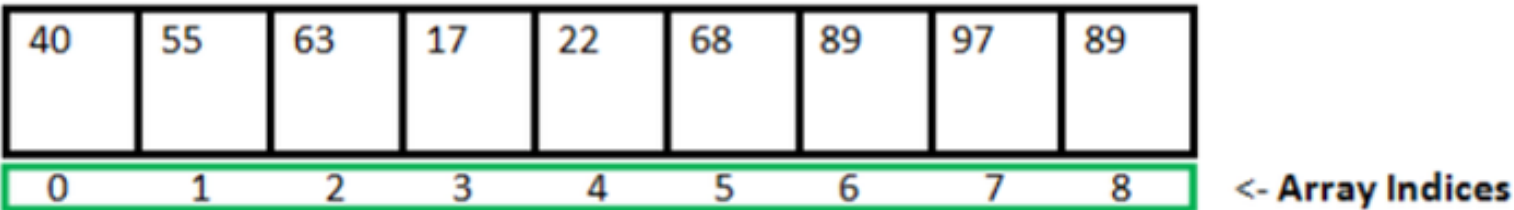
Fixed-size collection of same-type elements in contiguous memory, accessed by index.

KEY OPS

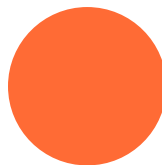
- Access by index
- Search by value
- Insert / delete (shifts elements)

ADV / DISADV

- + Fast constant-time access
- Fixed size, costly insert/delete



Array Length = 9
First Index = 0
Last Index = 8



Linked List

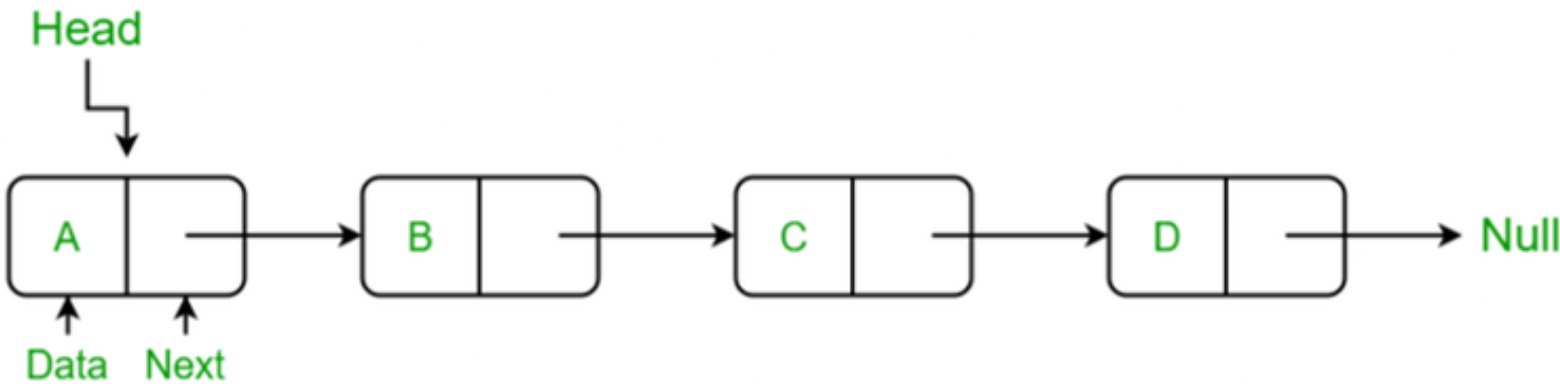
Dynamic chain of nodes, each holding data and a pointer to the next node.

KEY OPS

- Traversal head to tail
- Insertion / deletion via pointers

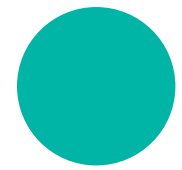
ADV / DISADV

- + Efficient dynamic memory use
- No direct index access; extra pointer memory



Stacks & Queues

Ordered access with opposite retrieval rules LIFO and FIFO.



Stack

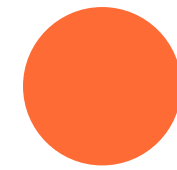
Last element added is the first removed. Access limited to the top.

KEY OPS

- Push : add to top
- Pop : remove from top
- Peek : view top item

ADV / DISADV

- + Fast for undo & expression parsing
- Only the top element is reachable



Queue

First element added is the first removed, using front and rear pointers.

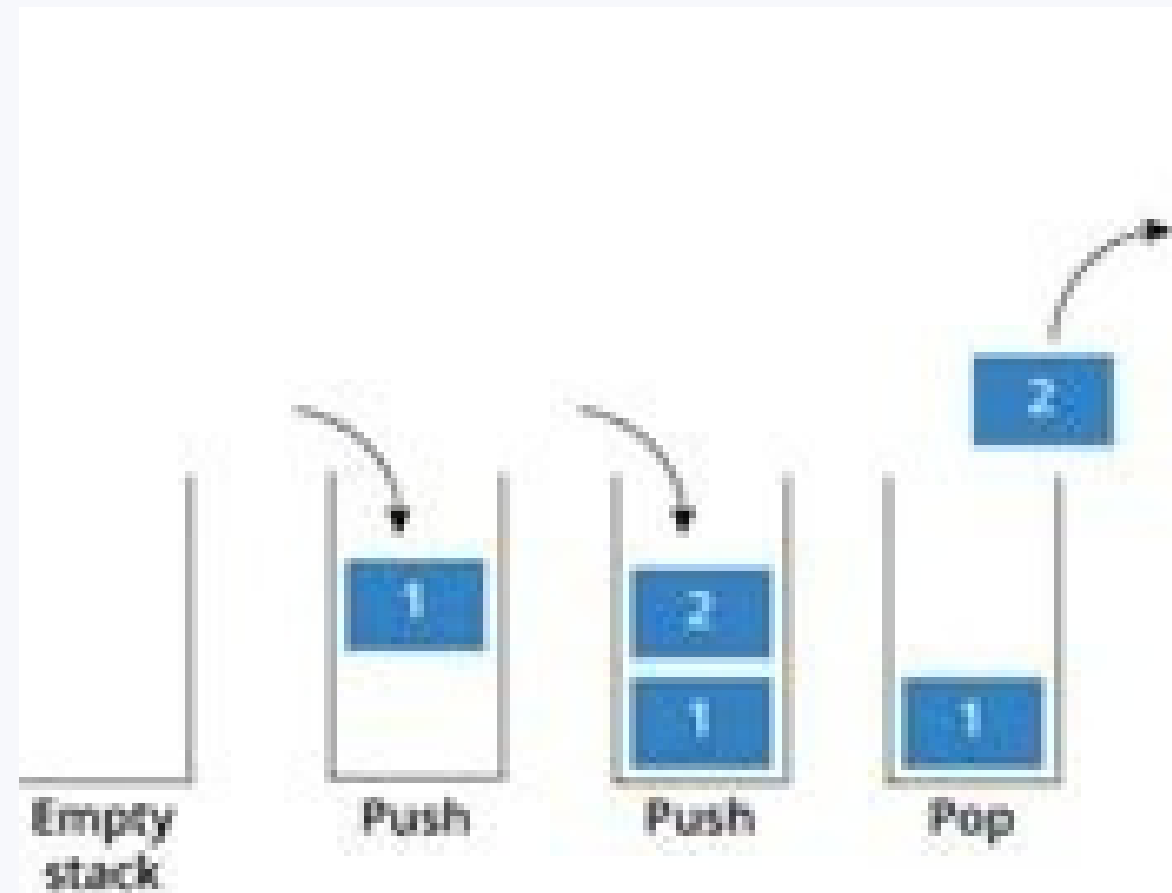
KEY OPS

- Enqueue : add at rear
- Dequeue : remove from front
- Peek : view front item

ADV / DISADV

- + Predictable, ordered processing
- Inefficient random access

Stacks & Queues



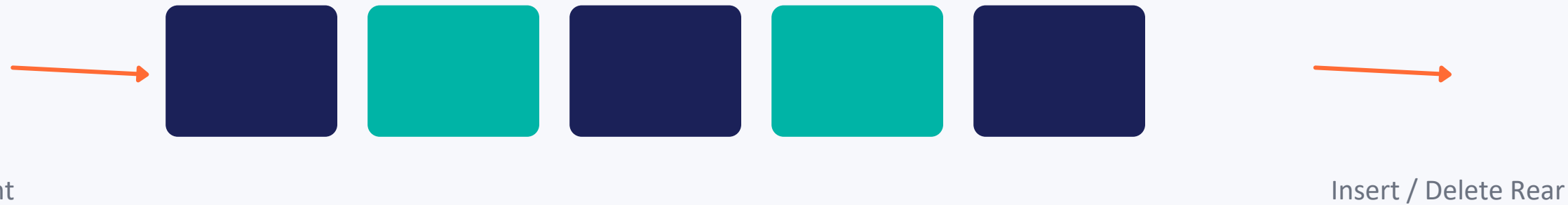
Stack



Queue

Deque : Double-Ended Queue

A queue that allows insertion and removal from both ends.



OPERATIONS

Insert Front / Rear
Delete Front / Rear

APPLICATIONS

Sliding window problems
Caching algorithms

TRADE-OFF

More flexible than queues
but more complex to build

Trees

A hierarchical structure of nodes, rooted at the top, with no cycles.

Characteristics

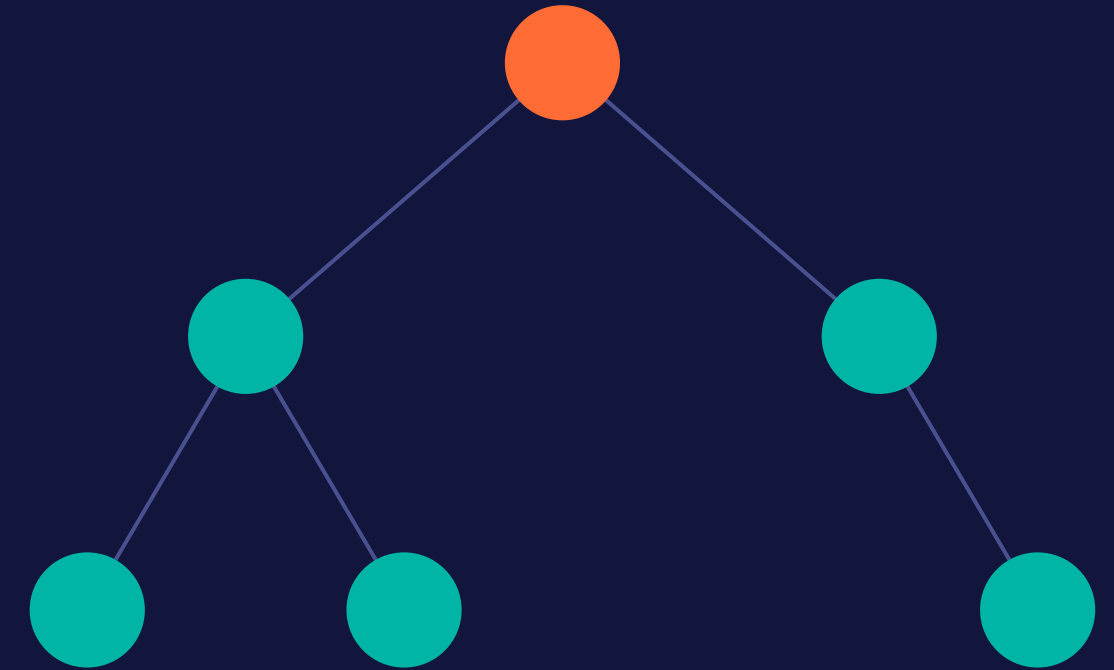
Root, parent & child nodes acyclic, recursive structure.

Applications

File systems, decision trees,
binary search trees.

Trade-off

Efficient search & sort when balanced
complex to implement.



Graphs

Vertices connected by edges, modeling relationships directed or undirected.

Characteristics

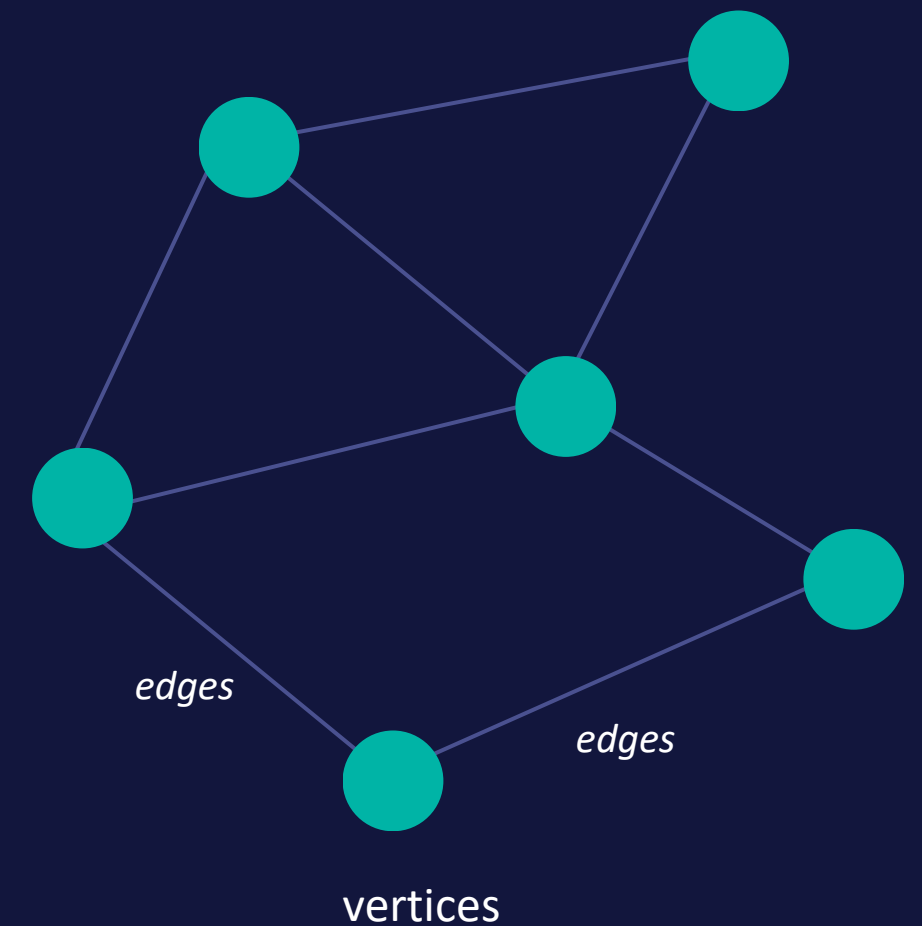
Vertices & edges , cyclic or acyclic, directed or undirected.

Applications

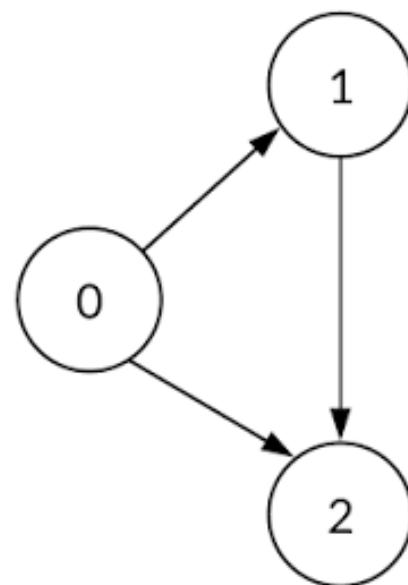
Social networks, shortest-path routing, web linking.

Trade-off

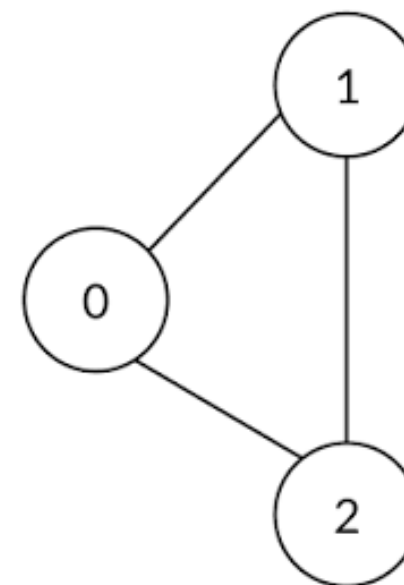
Models complex relationships
Memory-intensive at scale.



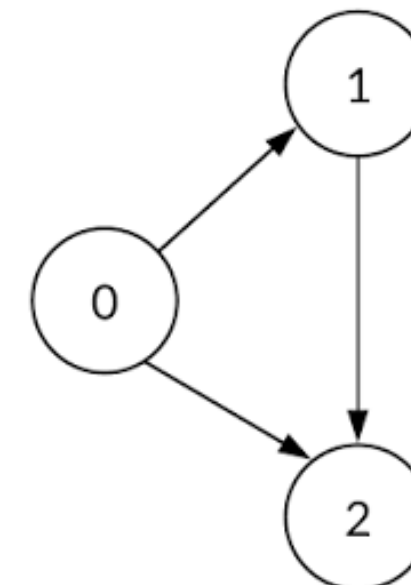
Directed Graph



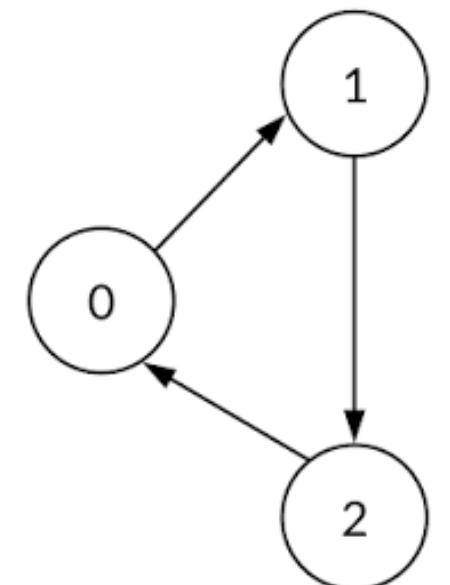
Undirected Graph



Acyclic Graph



Cyclic Graph



Hash Tables

Maps keys to values with a hash function for fast, associative lookup.



OPERATIONS

Insert : store a key-value pair
Search : look up by key
Delete : remove a pair

APPLICATIONS

Caching frequently used data
Database indexing
Associative arrays

TRADE-OFF

Constant-time average access
collisions & resizing cost efficiency

Linear vs. Non-Linear

Aspect	Linear	Non-Linear
Arrangement	Stored one after the other	Stored in a hierarchical manner
Access pattern	Single-pass traversal	Cannot traverse in a single pass
Time complexity	Grows with data volume	Stays steady as volume grows
Memory usage	Less efficient	More efficient
Examples	Array, Stack, Queue, Linked List	Tree, Graph, Hash Table

The Full Picture

1

Primitive

Integer, Float, Char, Boolean, String, Pointer

2

Linear

Array, Linked List, Stack, Queue, Deque

3

Non-Linear

Tree, Graph, Hash Table

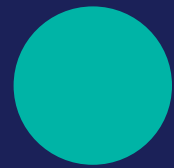
Useful Links



<https://github.com/gibsjose/cpp-cheat-sheet/blob/master/Data%20Structures%20and%20Algorithms.md>



<https://devdocs.io/cpp/>



<https://explainshell.com>

